SIMATS ENGINEERING

ASSESSMENT TOOL 1

SCENARIO BASED ASSIGNMENT

Course Name: Artificial Intelligence

Course Code: CSA17

Course Outcome: CO2 – Search and Game-Solving Techniques

Submitted by

Dinesh Prakasam

Register Number: 1921260184

Topics: Greedy Best-First Search, A*, Optimization, Online Search, CSP, Minimax and Alpha-Beta Pruning

1. AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the next location using only the heuristic estimate h(n), here the straight-line distance from the drone's current position to the destination. It chooses the point that appears closest to the medicine recipient. This can produce a quick route and is useful when a fast decision is important. However, it ignores the actual cost already spent, changing weather, terrain-related movement costs and some risks. A route that looks short may pass through dangerous weather or a blocked area, so the algorithm is not guaranteed to find the lowest-cost or safest route.

ii. How A* balances cost and heuristic

A* evaluates each candidate location using f(n) = g(n) + h(n). Here, g(n) is the known cost from the starting point to n, including travel time and other represented costs, while h(n) estimates the remaining cost to the destination. By considering both the cost already incurred and the estimated cost remaining, A* avoids choosing a route solely because it looks geographically close. If the heuristic never overestimates the true remaining cost and edge costs are modelled correctly, A* can find an optimal route in a static search problem.

iii. Impact of heuristic accuracy

A good heuristic guides the search toward the destination and can reduce the number of locations explored. A straight-line estimate is useful for distance, but it may be inaccurate when rivers, floodwater, wind, terrain or restricted airspace affect travel. Greedy Best-First Search is especially sensitive because it relies only on h(n). A* is also affected by the heuristic, but its g(n) term helps it account for the cost of the route already travelled. An overestimating heuristic can cause A* to lose its optimality guarantee.

iv. Effect of blocked paths and dynamic changes

When a path becomes blocked, the route information must be updated and the search should be run again or repaired. The drone may need to discard an invalid planned route and consider alternatives. Frequent changes increase computation and can make an old plan unsafe. The system should use the latest sensor and weather updates, check route safety before each movement, and include a fallback or return-to-safe-location plan.

v. Recommendation

A* is the stronger default when the drone must balance travel cost, delay and risk, provided that the cost model and heuristic are suitable. For real-time use, it should be combined with frequent updates, safety constraints and route replanning. If computation is extremely limited, a fast heuristic method can provide an initial route, but the drone must still validate that route against current hazards. Safety should take priority over a small reduction in travel time.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem

The system can be formulated as minimizing a total objective such as J = w■W + w■F + w■C, where W is total waiting time, F is fuel consumption, C is a congestion measure, and w■, w■ and w■ represent the importance assigned to each goal. Decision variables may include each intersection's green-light duration, phase order and coordination offsets. Constraints include minimum and maximum green times, safe amber/all-red intervals, pedestrian crossing requirements and conflict-free signal phases. Traffic counts and queue lengths are updated continuously.

Why traditional search is inefficient

With hundreds of intersections, many possible timings and changing traffic conditions, the number of combinations becomes extremely large. Exhaustively searching all combinations takes too long, and a solution based on old traffic data may quickly become unsuitable. A practical system needs heuristic optimization, local adjustments and repeated evaluation using current traffic measurements.

i. Hill Climbing and its limitations

Hill Climbing starts with a signal plan and repeatedly makes a small change, such as increasing one green phase or adjusting an offset. If the change improves the objective, it is retained. It is simple and can improve a plan quickly, but it usually accepts only locally better moves. Therefore it can stop at a solution that is better than nearby alternatives but not the best overall solution. It can also struggle when improvements require several coordinated changes.

ii. Local maxima, plateaus and ridges

A **local optimum** occurs when nearby timing changes do not improve the score, even though a better city-wide plan exists. A **plateau** is a broad region where many timing changes produce nearly the same score, giving the algorithm little direction. A **ridge** occurs when the best improvements require coordinated changes across several intersections; changing only one intersection may not help. In real traffic, these issues may appear when adjacent signals must be synchronized to clear a queue.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution early in the search, with a probability controlled by a temperature parameter. As the temperature decreases, the algorithm becomes less willing to accept worse moves. This helps it escape local optima. A Genetic Algorithm maintains a population of candidate signal plans and uses selection, crossover and mutation to generate new candidates. It explores several possibilities at once, although it can require more computation and careful parameter tuning.

iv. Recommendation

A hybrid approach is suitable: use a Genetic Algorithm or offline optimization to develop coordinated plans, then apply fast local search or Hill Climbing for small real-time corrections. Simulated Annealing can be useful when escaping local optima is important. Hard safety constraints must never be violated, and any selected plan should be checked against current traffic and pedestrian needs. The best method depends on the available computing time and the rate at which traffic changes.

3. Autonomous Mars Rover: Online Search

i. Why an online search agent is needed

An offline search agent plans using a map and transition information known in advance. A Mars rover does not have a complete map of every rock, crater or slope. It discovers terrain while moving and receives new sensor readings along the way. An online search agent can choose an action, observe the result, update its knowledge and then choose the next action. This makes it appropriate for exploration in an unknown environment.

ii. Partial observability and dynamic-environment challenges

The rover's sensors have limited range and may not detect hazards hidden behind rocks or beyond a ridge. Sensor noise can also make terrain estimates uncertain. The rover must balance progress toward sample locations with the risk of getting stuck, damaging equipment or losing communication. Conditions such as lighting, dust or wheel traction can affect movement. Because the full state is not known, decisions should account for uncertainty and maintain a safe margin around hazards.

iii. Building and updating the internal model

The rover combines camera images, range measurements, wheel odometry and other available sensor data to create a local map. It records traversable areas, hazards, estimated slopes and possible routes. After each movement, it compares the expected result with the observed result and updates its position estimate and map. Uncertain areas should be marked as uncertain rather than treated as definitely safe. The rover can then replan from its updated state.

iv. Online search compared with uninformed and informed search

Uninformed methods such as breadth-first search do not use a goal-distance heuristic and often require exploring many states. Informed methods such as A* use a heuristic to direct search toward a goal, but usually work best when the relevant map and costs are available. Online search makes decisions during exploration and learns about the environment as it goes. It can use informed heuristics where available, while still reacting to newly discovered obstacles.

v. Recommended approach

Use an online, receding-horizon strategy: plan over the known local map, move a limited distance, collect new sensor data and replan. Include hazard costs, uncertainty margins, energy limits and a safe fallback route. A* or another heuristic planner can guide movement in known regions, while the online agent handles newly discovered terrain. This approach is adaptable and safer than committing to a complete route based on an incomplete map.

4. University Exam Timetabling as a Constraint Satisfaction Problem

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, domains of possible values and constraints that restrict combinations of values. For exam timetabling:

• **Variables:** each course's exam time slot and exam hall. A combined variable can represent the pair (time slot, hall).
• **Domains:** the permitted time slots and halls for each course, excluding times when required faculty or facilities are unavailable.
• **Constraints:** a student cannot have two exams at the same time; a hall cannot host more than one exam in a slot; hall capacity and accessibility requirements must be met; prerequisite ordering rules must be followed; faculty availability must be respected; and special accommodations must be provided.

i. Explaining the constraints

If two courses share students, their exams must be assigned different time slots. Each hall has a capacity, so the number of students assigned to it cannot exceed that limit. If one subject must be held before another, the assigned time for the first must precede the second. A course requiring an accessible room must be assigned to a suitable hall. These conditions define which timetable assignments are valid.

ii. Backtracking search

Backtracking assigns a value to one variable at a time. After assigning an exam to a slot and hall, the system checks whether the assignment violates any constraint. If it is valid, the search proceeds to the next exam. If a conflict occurs and no value remains for the current variable, the algorithm returns to an earlier variable and tries a different assignment. Variable-ordering heuristics, such as choosing the exam with the fewest legal options first, can reduce unnecessary exploration.

iii. Constraint propagation and forward checking

Forward checking removes values from the domains of unassigned variables that have become impossible after a choice. For example, scheduling a course at 10 a.m. removes that slot from the domains of other courses that share students. If any unassigned course has no legal time left, the algorithm detects the conflict immediately and backtracks instead of exploring a doomed partial timetable. Propagation therefore reduces wasted search.

iv. Real-world challenges and scalable strategy

Large institutions may have thousands of students, courses, halls and special requirements. Data can change when students enroll, faculty availability changes or rooms become unavailable. A practical strategy combines the CSP model with the most-constrained-variable heuristic, least-constraining-value ordering, forward checking and conflict-directed backtracking. The system should validate the final timetable against every hard constraint, while treating preferences such as avoiding late exams as soft constraints to optimize.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the game

Minimax represents possible game decisions as a tree. The AI is the maximizing player, trying to choose the move with the highest utility, while the human opponent is modelled as the minimizing player, choosing moves that reduce the AI's utility. The algorithm evaluates terminal states or depth-limited states and propagates values upward: maximizing nodes take the largest child value, and minimizing nodes take the smallest. The AI selects the move with the best worst-case outcome, assuming both players play optimally.

ii. Computational challenges of large game trees

If the average number of legal moves is b and the search depth is d, a basic Minimax search may examine on the order of O(b^d) nodes. The tree grows rapidly as either the number of possible moves or the depth increases. In a real-time strategy game, the AI must also account for many interacting units, resources and changing events. Searching the entire tree may exceed the time limit, so the system needs pruning, depth limits and an evaluation function.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning gives the same Minimax result but avoids exploring branches that cannot affect the final choice. Alpha is the best value the maximizing player can guarantee so far; beta is the best value the minimizing player can guarantee. At a maximizing node, alpha is updated when a better value is found. At a minimizing node, beta is updated when a lower value is found. When alpha is greater than or equal to beta, the remaining children of that node can be skipped because they cannot change the decision. Good move ordering increases pruning; in the best case, the effective search depth can be substantially increased within the same time.

iv. Example evaluation function

A simple depth-limited evaluation function from the AI's perspective could be:
Score = 5(Unit strength difference) + 3(Resource advantage) + 4(Objective control difference) + 2(Map control difference) – **6(Immediate threat to critical base)**.
The coefficients are illustrative weights, not universal values. Unit strength measures remaining combat power; resources estimate the ability to produce more units; objective control rewards progress toward victory; map control reflects strategic position; and the threat term penalizes danger to the main base. The function should be tuned and tested for the specific game, and terminal win/loss states should receive much larger positive/negative values.

v. Recommendation for real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, ordering promising moves first and using a fast evaluation function at the search limit. Iterative deepening can produce a usable move even if the time limit interrupts a deeper search. The AI can allocate more search time to critical decisions and less to routine ones. This provides a practical balance between decision quality and response speed, although it does not guarantee a globally optimal move when the search is cut off at a limited depth.

Conclusion

Search and game-solving methods help AI systems make decisions in large and uncertain problem spaces. Greedy Best-First Search is fast but can overlook route cost, while A* balances the known path cost with an estimate of the remaining cost. Optimization methods such as Simulated Annealing and Genetic Algorithms can help with complex traffic-signal timing. Online search supports safe exploration when a complete map is unavailable. CSP techniques organize exam timetabling constraints, and Minimax with Alpha-Beta pruning supports strategic game decisions under time limits. In every scenario, the algorithm should be selected according to the problem's constraints, data quality, safety requirements and available computation.

Student: Dinesh Prakasam
Register Number: 1921260184